

Chương 1

Lập trình C++ cơ bản

Nội dung

- 1.1 Chương trình C++
- 1.2 Kiểu dữ liệu, biểu thức và phép toán
- 1.3 Lệnh điều khiển
- 1.4 Kiểu dữ liệu mảng
- 1.5 Kiểu dữ liệu cấu trúc
- 1.6 Hàm

1.1 Chương trình C++

A) Giới thiệu ngôn ngữ C++

- C++ là ngôn ngữ lập trình máy tính bậc cao do Bjarne Stroustrup phát minh năm 1979 dựa trên ngôn ngữ lập trình C
- Nhờ ưu điểm đa năng, mạnh mẽ và hỗ trợ lập trình hướng đối tượng, C++ trở nên phổ biến và thành tiêu chuẩn quốc tế ISO 14882
 - C++11, C++14, C++17
 - Luôn trong top 5 ngôn ngữ phổ biến nhất thế giới

- C++ được 1 loạt ngôn ngữ bậc cao hiện đại tham khảo và kế thừa như Java, Javascript, PHP, C#, Objective-C
- C++ thường được giảng dạy trong các ngành kỹ thuật nói chung và ngành máy tính nói riêng

B) Cấu trúc chương trình C++

- Chương trình C++ gồm dãy các câu lệnh được tổ chức để thực hiện các thao tác tính toán trên máy tính
- Về hình thức, chương trình C++ được lưu trong tệp tin mã nguồn có phần mở rộng là .cpp
- Về nội dung, chương trình tuân thủ theo cú pháp của ngôn ngữ, có cấu trúc và có ngữ nghĩa

Cấu trúc tổng quát của một chương trình C++

Chỉ thị include để chèn các tệp tiêu đề của thư viện lập trình

```
#include <...>
```

Phần khai báo và định nghĩa các kiểu dữ liệu, các hàm, các biến và hằng dữ liệu sẽ được sử dụng trong chương trình

Chương trình chính – hàm **main()**

```
int main()  
{  
    các_câu_lệnh;  
    return 0;  
}
```

Chương trình Hello World

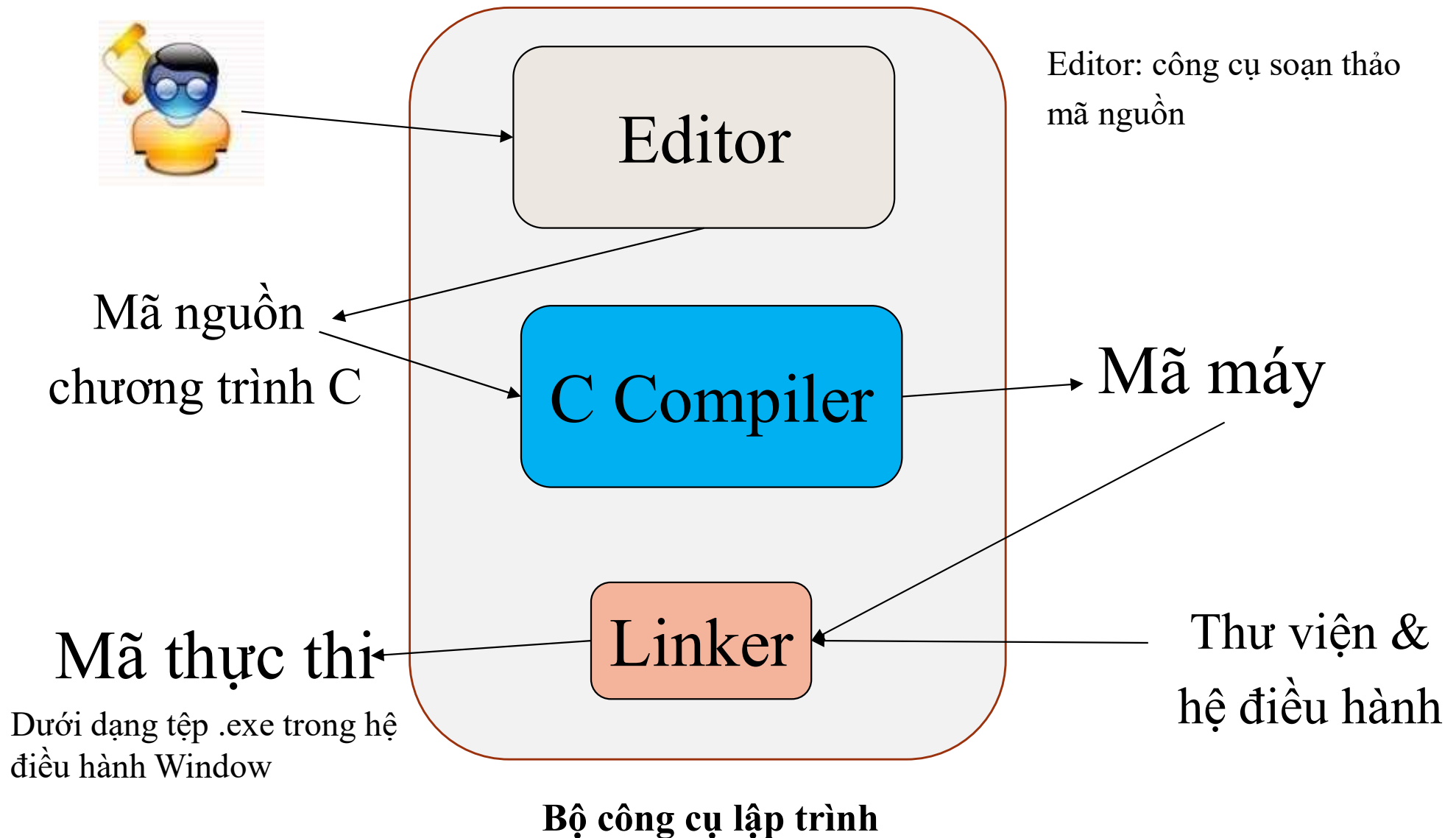
```
/*  Chuong trinh Hello World
    helloworld.cpp
*/
#include <iostream>

int main()
{
    std::cout << "Hello World" << endl;
    return 0;
}
```

C) Biên dịch và liên kết chương trình

- Chương trình Hello World được gọi là mã nguồn hay **source code**
- Để máy tính “hiểu” được chương trình thì cần dịch mã nguồn sang mã máy (**machine code**) bằng trình biên dịch (**compiler**)
- Để tạo ra chương trình “chạy” được, cần liên kết mã máy và tạo ra mã thực thi (**executable code**) bằng trình liên kết (linker)

D) Bộ công cụ lập trình (IDE)



Giới thiệu bộ công cụ CodeBlock

- Bộ công cụ lập trình còn gọi là môi trường phát triển tích hợp (Integrated Development Environment – IDE)
- Bao gồm 3 công cụ chính: editor, compiler, linker và nhiều tool khác hỗ trợ công việc lập trình, phát triển ứng dụng

1.2 Kiểu dữ liệu, biểu thức và phép toán

- Kiểu dữ liệu cơ bản được thiết kế sẵn trong ngôn ngữ lập trình, gồm có:
 - Kí tự : **char, wchar_t**
 - Số nguyên: **int**
 - Số thực: **float, double**
 - logic: **bool**
- Kiểu dữ liệu dẫn xuất là kiểu mở rộng từ kiểu cơ bản
 - **short int, long int, long double**
 - **unsigned char, unsigned int**
 - **size_t**

Bảng kiểu dữ liệu cơ bản của C++

Kiểu dữ liệu	Ý nghĩa	Kích thước	Miền giá trị
unsigned char	Kí tự ASCII	1 byte	$0 \div 255$
char	Kí tự có dấu	1 byte	$-128 \div 127$
bool	Logic	1 byte	true, false
wchar_t	Kí tự Unicode	2 byte	$-32,768 \div 32,767$
unsigned int	Số nguyên không dấu	4 bytes	$0 \div 4,294,967,295$
short int	Số nguyên có dấu	2 bytes	$-32,768 \div 32,767$
int	Số nguyên có dấu	4 bytes	$-2,147,483,648 \div 2,147,483,647$
unsigned long	Số nguyên không dấu	4 bytes	$0 \div 4,294,967,295$
long	Số nguyên có dấu	4 bytes	$-2,147,483,648 \div 2,147,483,647$
float	Số thực dấu phẩy động, độ chính xác đơn	4 bytes	$\pm 3.4\text{E}-38 \div \pm 3.4\text{E}+38$
double	Số thực dấu phẩy động độ chính xác kép	8 bytes	$\pm 1.7\text{E}-308 \div \pm 1.7\text{E}+308$
long double	Số thực dấu phẩy động	10 bytes	$\pm 3.4\text{E}-4932 \div \pm 1.1\text{E}+4932$

Khai báo dữ liệu trong chương trình

- Khai báo biến dữ liệu với tên kiểu và tên biến

```
char c_1, c_2 ;
```

```
int i,n;
```

```
double x;
```

```
float a,b,c = 0.0; // khởi tạo giá trị cho biến c
```

- Khai báo hằng dữ liệu với từ khóa const, tên kiểu, tên hằng và giá trị

```
const int N = 10000;
```

```
const float PI = 3.14159;
```

```
const double T = 1.275E+6;
```

Nhập giá trị cho biến dữ liệu

- Luồng dữ liệu vào (input stream object): **cin**
- Thư viện: **#include <iostream>**
- Sử dụng không gian tên chuẩn: **using namespace std;**
- Sử dụng **cin** với phép toán nhập **>>**

```
int n;  
cin >> n; // nhập giá trị cho biến n  
float a, b, c;  
cin >> a >> b >> c ; // nhập giá trị cho  
a, b và c
```

Lệnh gán và biểu thức

- Lệnh gán giá trị cho biến

```
int j, k, i = 0;  
      k = i + 1; // gán giá trị i+1  
cho biến k  
      j = i = k + 2; // gán giá trị k + 2  
cho i và cho j
```

- Các câu lệnh tính toán được viết dưới dạng biểu thức gồm các toán hạng và phép toán

$i + 1$ là biểu thức với phép $+$, toán hạng i và 1

$j = i = k + 2$ là biểu thức với phép $=$, $+$ và toán hạng j , i , k

Biểu thức số học

- Toán hạng phải có kiểu số (char, int, long, float, double)
- Phép toán số học: + - * / %
- Chú ý phép chia số thực khác phép chia số nguyên

```
int i = 3, j;  
float t;  
j = i / 2;      // j = 1  
t = i / 2.0;    // t = 1.5
```


Biểu thức logic

- Giá trị chỉ có **true** hoặc **false**
- Phép toán so sánh: `>` `<` `>=` `<=` `==` `!=`
- Phép toán logic: `&&` `||` `!`
- Chú ý giá trị 0 được coi tương đương giá trị **false**
 - `0 && a > 1` nhận giá trị **false**
 - `1 || b < 2` nhận giá trị **true**
 - `2 && 4 > 2` nhận giá trị **true**
- Được sử dụng trong các cấu trúc lệnh có điều kiện như `if`, `while`, `for`

Các phép toán viết tắt

- Phép tăng 1 đơn vị: ++

```
int i = 0, k;
```

```
i++; // i = 1
```

```
k = i++; // k = 1 và i = 2
```

```
k = ++i; // k = 3 và i = 3
```

- Phép giảm 1 đơn vị: --

- Phép tính kết hợp phép gán

```
i += 2 ; // i = i + 2
```

```
i *= 2 ; // i = i * 2
```

Thứ tự ưu tiên của phép toán

1) Phép toán 1 ngôi: nếu 2 phép toán cùng mức thì ưu tiên thực hiện từ phải qua trái

$++$, $--$, $-$, $!$, $\&$

2) Phép toán 2 ngôi: nếu 2 phép toán cùng mức thì ưu tiên thực hiện từ trái qua phải

- Phép số học: $*$, $/$, $\%$
- Phép số học: $+$, $-$
- Phép so sánh lớn hơn nhỏ hơn: $>$, $<$, $>=$, $<=$,
- Phép so sánh bằng: $==$, $!=$
- Phép logic AND: $\&\&$
- Phép logic OR: $||$

In giá trị biểu thức ra màn hình

- Luồng dữ liệu ra (output stream object): **cout**
- Thư viện: **iostream, iomanip**
- Sử dụng không gian tên chuẩn: **using namespace std;**
- Sử dụng **cout** với phép toán in ra màn hình **<<**

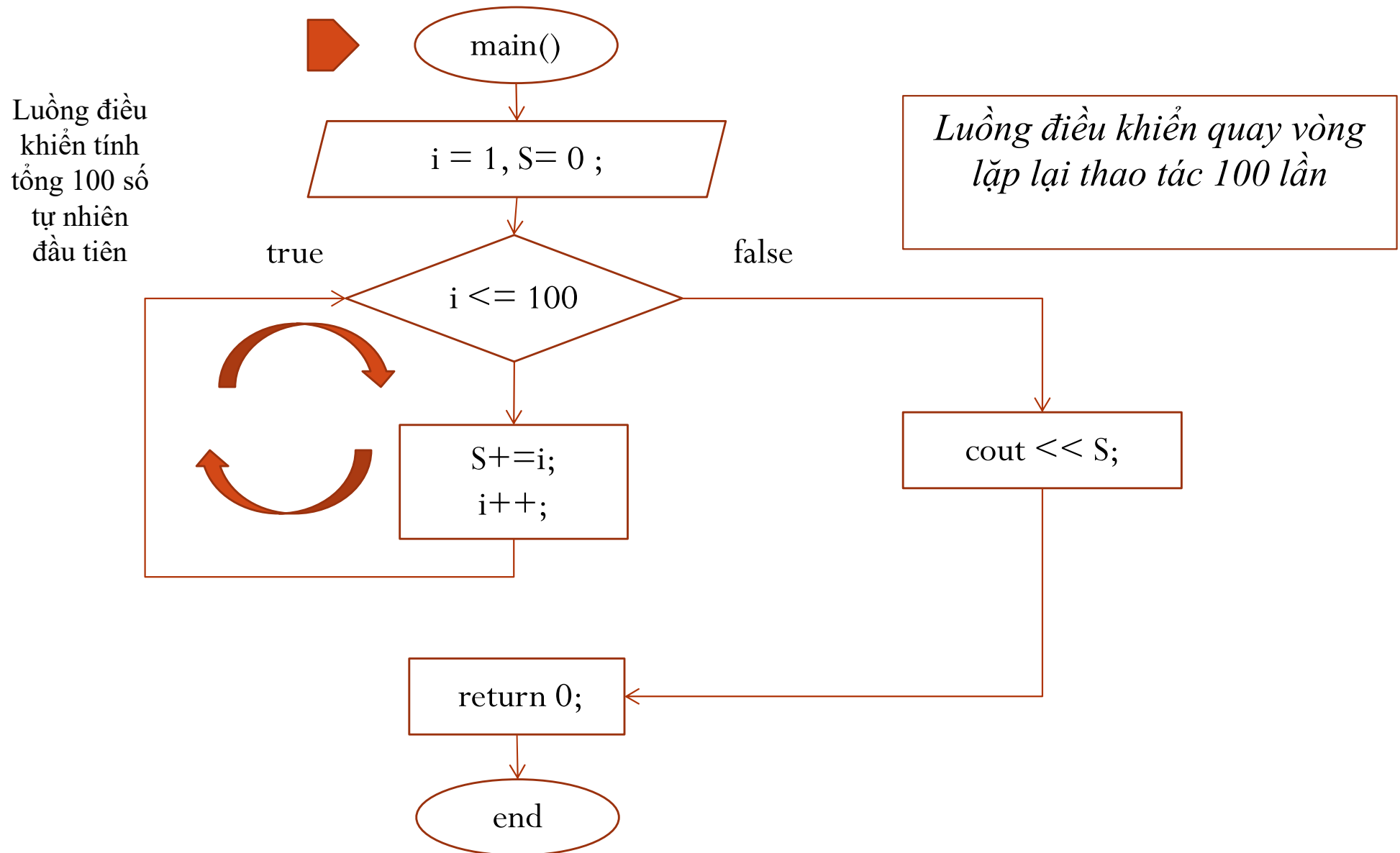
```
int i;  
float x;  
cin >> i; // nhập giá trị cho biến i  
cout << setw(3) << i << setw(5) << i++ << setw(5);  
cout << fixed << setprecision(4) << i *  
2.0 << setw(8);  
cout << scientific << left << x *= 100 << endl;
```

Định dạng xuất dữ liệu

- **setw(n)** đặt độ rộng dành cho dữ liệu
- **setprecision(n)** đặt độ chính xác bao nhiêu chữ số thập phân cho số thực, sử dụng cùng với
 - **fixed**: định dạng dấu chấm tĩnh
 - **scientific**: dấu chấm động và đi kèm số mũ
- **showpoint** - luôn hiển thị dấu chấm thập phân
- **left** căn lề trái
- **right** căn lề phải (mặc định)

Bài tập minh họa

1.3) Lệnh điều khiển



A) Cấu trúc lệnh chu trình lặp

- Cấu trúc lệnh `for()`

```
for(i=1, S=0; i<=100;i++)
```

```
S+=i;
```

- Cấu trúc lệnh `while()`

```
i=1; S =0;
```

```
while (i<=100) {
```

```
S+=i;
```

```
i++;
```

```
}
```

`do while()`

Cấu trúc chu trình lồng nhau

- Trường hợp với 3 cấu trúc lệnh for

```
for(i=0, count=0; i<3; i++) // chu trình ngoài cùng
{
    for(j=0; j<3; j++) // chu trình giữa
    {
        for(k=0; k<3; k++) // chu trình trong cùng
        {
            cout << i << "," << j << "," << k << endl;
            count++;
        }
    }
}
```

Lệnh continue và lệnh break

- Lệnh **continue**;
 - Đặt bên trong chu trình
 - Tác dụng: ngay lập tức nhảy đến lần lặp tiếp theo của chu trình, bỏ qua các câu lệnh đứng sau **continue**
- Lệnh **break**;
 - Đặt bên trong chu trình
 - Tác dụng: nhảy ra khỏi chu trình, kết thúc vòng lặp vô điều kiện.

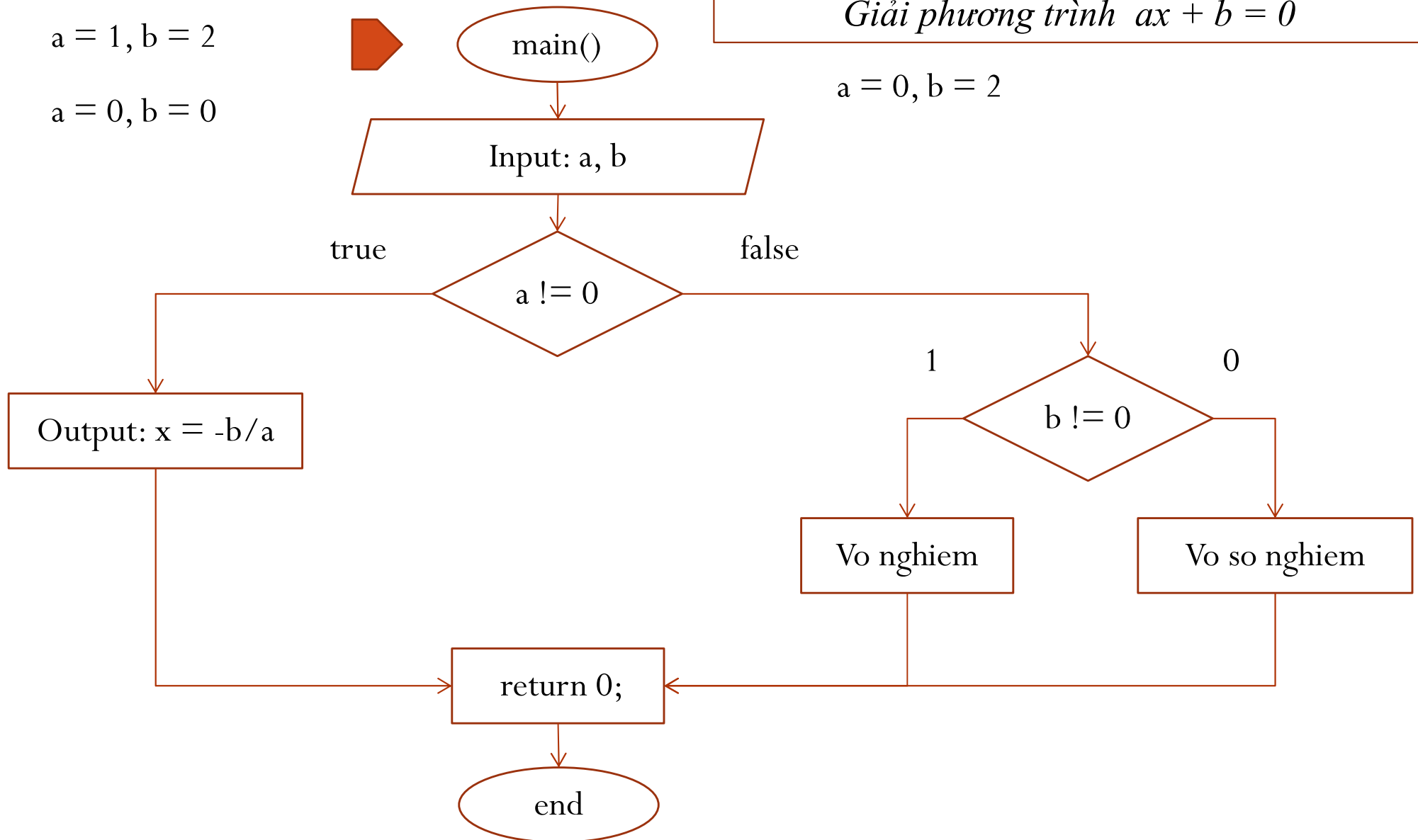
B) Cấu trúc lệnh rẽ nhánh

$a = 1, b = 2$

$a = 0, b = 0$

Giải phương trình $ax + b = 0$

$a = 0, b = 2$



Nhiều cấu trúc if else lồng nhau

- Giải phương trình bậc 1

```
if (a!=0) {  
    cout<<"Nghiem duy nhat x = ", -b/a);  
}  
else {  
    if(b!=0)  
        cout<<"Phuong trinh vo nghiem\n";  
    else  
        cout<<"Phuong trinh vo so nghiem\n";  
}
```

B.1) Cấu trúc lệnh if else

- Cú pháp 1: **if** (biểu_thức) lệnh;
- Cú pháp 2: **if** (biểu_thức) lệnh_1;
else lệnh_2;
- Chú ý: biểu_thức phải có kiểu số nguyên hoặc logic và đặt trong ngoặc ()

B.2) Cấu trúc lệnh switch

```
switch(biểu_thức)
{
    case giá_trị_1:  lệnh_1; break;
    case giá_trị_2:  lệnh_2; break;
    ...
    case giá_trị_n:  lệnh_n; break;

    [default: lệnh_n+1; break;]
}
```

Mệnh đề **default** có thể vắng mặt trong lệnh

Trình tự thực hiện lệnh switch case

- 1) Tính giá trị của biểu_thức (kiểu số nguyên)
- 2) Thực hiện rẽ nhánh
 - Nếu biểu_thức == giá_trị_1: chuyển đến lệnh 1
 - ...
 - Nếu biểu_thức == giá_trị_n: chuyển đến lệnh n
 - Nếu biểu_thức có giá trị khác: chuyển đến default

Ví dụ

• `diem_HP = 'B';` // Quy đổi điểm từ chữ sang số

```
➡ switch(diem_HP)
{
    case 'A': printf("Diem so: 4.0"); break;
    case 'B': printf("Diem so: 3.0"); break;
    case 'C': printf("Diem so: 2.0"); break;
    case 'D': printf("Diem so: 1.0"); break;
    case 'F': printf("Khong dat"); break;
}
<lệnh tiếp>;
```


Chú ý với lệnh break;

- Lệnh break dùng để ngắt luồng điều khiển và thoát ra ngoài cấu trúc lệnh switch
- Nếu thiếu break, thì luồng điều khiển sẽ thực hiện các lệnh tiếp bên trong cấu trúc

Ví dụ

- Thiếu lệnh break

```
• Diem_HP = 'B';
```

```
 switch(diem_HP)
{
    case 'A': printf("Diem so: 4.0"); break;
    case 'B': printf("Diem so: 3.0"); // thiếu break
    case 'C': printf("Diem so: 2.0"); break;
    case 'D': printf("Diem so: 1.0"); break;
    case 'F': printf("Khong dat: 0"); break;
}
<lệnh tiếp>;
```

Kết quả: Diem so: 3.0Diem so 2.0